Practical : 2	Breadth-First Search (BFS) on a Graph Using Python..
Date : 10/08/26	

Aim: -

To implement Breadth-First Search (BFS) on a graph using Python and traverse the graph level by level.

Algorithm:-

1. Start the program.
2. Import the required libraries.
3. Create the graph using a dictionary.
4. Select the starting node as A.
5. Create an empty queue and an empty visited list.
6. Add the starting node to the queue and mark it as visited.
7. Remove the first node from the queue.
8. Visit all unvisited neighbouring nodes of the current node.
9. Add each unvisited neighbour to the queue and mark it as visited.
10. Repeat the process until the queue becomes empty.
11. Display the final BFS traversal.
12. Stop the program.

Program:-

```
# ================================================================
# Practical No. 2
# Breadth-First Search (BFS)
# ================================================================

from collections import deque
import time
import os
import datetime

# Display Program Title
print("=" * 65)
print(" BREADTH FIRST SEARCH (BFS)")
print("=" * 65)

# Display Current Date and Time
current = datetime.datetime.now()
print("\nDate :", current.strftime("%d-%m-%Y"))
print("Time :", current.strftime("%H:%M:%S"))

# Display Operating System
print("\nOperating System :", os.name)

# Graph Creation
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"],
    "D": [],
    "E": [],
    "F": [],
    "G": []
}

# Display Graph
print("\nGraph Structure")
for node in graph:
    print(node, "->", graph[node])
```

```
# Create Empty Queue and Visited List
queue = deque()
visited = []

# Starting Node
start_node = "A"

print("\nStarting Node :", start_node)

# Add Starting Node into Queue
queue.append(start_node)

# Mark Starting Node as Visited
visited.append(start_node)

print("\nQueue :", list(queue))
print("Visited :", visited)

print("\nAI is Preparing BFS Traversal...")
time.sleep(2)

print("\nTraversal Started...\n")

# BFS Loop
while queue:
    # Remove first element (FIFO)
    current_node = queue.popleft()

    print("-------------------------")
    print("Current Node :", current_node)
    print("Queue After Removal :", list(queue))

    print("Checking Neighbor Nodes...")
    time.sleep(1)

    # Visit every neighbor
    for neighbour in graph[current_node]:
        if neighbour not in visited:
            print("Found New Node :", neighbour)

            queue.append(neighbour)
            visited.append(neighbour)
```

```
            print("Added to Queue :", neighbour)
            print("Current Queue :", list(queue))
            print("Visited Nodes :", visited)

    print("-------------------------")
    print("Queue Status :", list(queue))
    time.sleep(1)

# BFS Completed
print("\n" + "=" * 65)
print(" BFS TRAVERSAL COMPLETED")
print("=" * 65)

print("\nFinal Visited Nodes")
for node in visited:
    print(node, end=" ")

print("\n\nTotal Nodes Visited :", len(visited))
print("\nTraversal Completed Successfully.")
print("=" * 65)
```

Output:

```
C:\Users\Vishnu\OneDrive\AI_Practicals>python practical2_bfs.py
======================================================================
 BREADTH FIRST SEARCH (BFS)
======================================================================

Date : 23-08-2026
Time : 22:41:38

Operating System : nt

Graph Structure
A -> ['B', 'C']
B -> ['D', 'E']
C -> ['F', 'G']
D -> []
E -> []
F -> []
G -> []

Starting Node : A

Queue : ['A']
Visited : ['A']

AI is Preparing BFS Traversal...

Traversal Started...

------------------------------------
Current Node : A
Queue After Removal : []
Checking Neighbor Nodes...
Found New Node : B
Added to Queue : B
Current Queue : ['B']
Visited Nodes : ['A', 'B']
Found New Node : C
Added to Queue : C
Current Queue : ['B', 'C']
Visited Nodes : ['A', 'B', 'C']
```

```
--------------------------------------
Queue Status : ['B', 'C']
--------------------------------------
Current Node : B
Queue After Removal : ['C']
Checking Neighbor Nodes...
Found New Node : D
Added to Queue : D
Current Queue : ['C', 'D']
Visited Nodes : ['A', 'B', 'C', 'D']
Found New Node : E
Added to Queue : E
Current Queue : ['C', 'D', 'E']
Visited Nodes : ['A', 'B', 'C', 'D', 'E']
--------------------------------------
Queue Status : ['C', 'D', 'E']
--------------------------------------
Current Node : C
Queue After Removal : ['D', 'E']
Checking Neighbor Nodes...
Found New Node : F
Added to Queue : F
Current Queue : ['D', 'E', 'F']
Visited Nodes : ['A', 'B', 'C', 'D', 'E', 'F']
Found New Node : G
Added to Queue : G
Current Queue : ['D', 'E', 'F', 'G']
Visited Nodes : ['A', 'B', 'C', 'D', 'E', 'F', 'G']
--------------------------------------
Queue Status : ['D', 'E', 'F', 'G']
--------------------------------------
Current Node : D
Queue After Removal : ['E', 'F', 'G']
Checking Neighbor Nodes...
--------------------------------------
Queue Status : ['E', 'F', 'G']
--------------------------------------
Current Node : E
Queue After Removal : ['F', 'G']
Checking Neighbor Nodes...
--------------------------------------
```

```
Queue Status : ['F', 'G']
------------------------------------
Current Node : F
Queue After Removal : ['G']
Checking Neighbor Nodes...
------------------------------------
Queue Status : ['G']
------------------------------------
Current Node : G
Queue After Removal : []
Checking Neighbor Nodes...
------------------------------------
Queue Status : []

====================================================================
 BFS TRAVERSAL COMPLETED
====================================================================

Final Visited Nodes
A B C D E F G

Total Nodes Visited : 7

Traversal Completed Successfully.
====================================================================
```

Conclusion:

Thus, the **Breadth-First Search (BFS) algorithm was successfully implemented using Python**. The graph was traversed starting from node A, and the nodes were visited level by level using a **Queue (FIFO)** data structure.